

Dakar Institute of Technology



Habilitation n°3386-21 FEV.2024

Domaine : Sciences et Technologies

Département : Informatique

Spécialité : Intelligence Artificielle

Master 2 IA : Examen Outils de Versioning

Projet : Bibliothèque Numérique avec Système de Recommandation

Présenté par

D. M. Maurice AHOUANSOU

M. DIAKITE

Abdoul G. DIALLO

Ibrahima DIALLO

Seydou KABORE

M. KANE

Abdin KOUSSUBE

Fatoumata Bintou SIDIBE

Année Académique : 2025 – 2026

SOMMAIRE

RÉSUMÉ	5
1 Généralités	6
1.1 Introduction	6
1.2 Problématique	6
1.3 Objectifs du projet	7
1.4 Structure du rapport	7
1.5 Stack technique	8
2 Architecture du Système	9
2.1 Vue d'ensemble de l'architecture microservices	9
2.2 Cartographie des ports	9
2.3 Structure du dépôt GitHub	10
2.4 Schéma de la base de données	11
3 Description des APIs	13
3.1 Service Livres — Port 3001	13
3.2 Service Utilisateurs — Port 3002	13
3.3 Service Emprunts — Port 3003	14
3.4 Service Recommandation — Port 3004 (FastAPI)	18
3.4.1 Logique de recommandation	18
4 Workflow Git Utilisé	19
4.1 Stratégie de branches (Gitflow)	19
4.2 Commandes Git avancées utilisées	19
4.3 Conventions de commits	20
5 Pipeline DVC et Métriques du Modèle	21
5.1 Initialisation et remote Google Drive	21
5.2 Structure du pipeline DVC	21
5.3 Fichier <code>dvc.yaml</code>	22
5.4 Hyperparamètres du modèle SVD	23
5.5 Reproduction et rôle de <code>dvc.lock</code>	23
5.6 Métriques du modèle	24
5.6.1 Affichage et comparaison	24
5.6.2 Tableau comparatif des versions	24

5.7	Versioning du modèle et chargement dans l'API	24
6	Réalisation du Projet	26
6.1	Environnement de travail	26
6.1.1	Docker et Docker Compose	26
6.1.2	Comptes de démonstration	28
6.1.3	Démarrage rapide	28
6.2	Pipeline CI/CD avec Jenkins (bonus)	29
6.3	Captures d'écran de l'application	30
6.3.1	Page d'accueil — Catalogue des livres	30
6.3.2	Gestion des emprunts	31
6.3.3	Recommandations personnalisées	32
6.3.4	Tableau de bord administrateur	33
6.3.5	Documentation Swagger — Service Recommandation	34
	Conclusion	36
	Bibliographie	37

Table des figures

2.1	Architecture microservices de la Bibliothèque Numérique DIT	9
2.2	Schéma de la BD relationnelle de la Bibliothèque Numérique DIT . .	12
3.1	Architecture du service emprunt	15
3.2	Architecture création emprunt	17
4.1	Diagramme du workflow Gitflow	19
5.1	Graphe de dépendances du pipeline DVC	22
6.1	Page de connexion à la bibliothèque	30
6.2	Page d'accueil — Catalogue de la bibliothèque	31
6.3	Interface de gestion des emprunts	32
6.4	Système de recommandation personnalisée	33
6.5	Tableau de bord administrateur	34
6.6	Documentation Swagger UI — Service Recommandation	35

Liste des tableaux

1.1	Technologies utilisées dans le projet	8
2.1	Services, ports et technologies	9
2.2	Tables de la base de données	11
3.1	Endpoints — Service Livres	13
3.2	Endpoints — Service Utilisateurs	14
3.3	Endpoints — Service Emprunts	16
3.4	Endpoints — Service Recommandation	18
5.1	Hyperparamètres de l’algorithme SVD (scikit-surprise)	23
5.2	Évolution des métriques du modèle SVD	24
6.1	Comptes pré-chargés dans la base de données	28

RÉSUMÉ

Le présent projet, intitulé **Bibliothèque Numérique DIT**, porte sur la conception et la mise en œuvre d'une plateforme complète de gestion de bibliothèque académique basée sur une architecture microservices. L'objectif principal était de développer un système moderne permettant la gestion des livres, des utilisateurs et des emprunts, enrichi d'un système de recommandation intelligente basé sur l'algorithme SVD (Singular Value Decomposition), le tout versionné et orchestré selon les meilleures pratiques DevOps et MLOps.

L'architecture développée repose sur cinq microservices indépendants : un service de gestion des livres, un service d'authentification des utilisateurs, un service de gestion des emprunts, un service de recommandation basé sur FastAPI et scikit-surprise, et une interface frontend en React 18. Chaque service dispose de son propre Dockerfile et communique via des API REST. L'ensemble est orchestré par Docker Compose avec une base de données PostgreSQL 15 partagée.

Le versioning des données et du modèle ML a été assuré avec DVC (Data Version Control), permettant un pipeline reproductible en trois étapes : prétraitement, entraînement et évaluation. Un remote Google Drive a été configuré pour stocker les artefacts. Le modèle SVD a été amélioré entre deux versions, passant d'un RMSE de 1.024 à 0.843, soit une amélioration de 17.68%.

Le versioning du code a suivi le workflow Gitflow avec des branches dédiées, des tags sémantiques et des commits conventionnels. Un pipeline CI/CD a été déployé avec Jenkins pour automatiser les étapes de construction, de test et de déploiement.

Ce projet démontre la pertinence des architectures microservices modernes appliquées à la gestion académique, et constitue une application concrète des compétences en développement backend, conteneurisation, machine learning et gestion de versions.

Chapitre 1:

Généralités

1.1 Introduction

À l’heure du numérique, les institutions académiques génèrent quotidiennement un volume croissant de données liées à leurs activités : gestion des ressources documentaires, suivi des emprunts, analyse des comportements des étudiants. Dans le secteur de la gestion de bibliothèque, chaque emprunt, retour ou consultation représente une source précieuse d’informations pouvant être exploitée pour améliorer les services offerts aux utilisateurs.

Face à cette réalité, les technologies modernes de développement web et de machine learning offrent aujourd’hui des solutions performantes pour concevoir des plateformes intelligentes, capables non seulement de gérer les opérations courantes, mais aussi de proposer des recommandations personnalisées basées sur l’historique des utilisateurs.

C’est dans ce contexte que s’inscrit le projet **Bibliothèque Numérique DIT**, dont l’objectif est de concevoir une plateforme web complète de gestion de bibliothèque académique, construite sur une architecture microservices, enrichie d’un système de recommandation basé sur l’apprentissage automatique, et soumise à une gestion avancée du versioning. À travers cette réalisation, il s’agit de mettre en pratique les concepts fondamentaux des outils de versioning — Git, Docker et DVC — dans un contexte applicatif complet et professionnel.

1.2 Problématique

Le Dakar Institute of Technology (DIT) gérait jusqu’à présent sa bibliothèque académique de façon entièrement manuelle. Cette approche engendré de nombreuses lacunes opérationnelles :

- ✓ **Difficulté de suivi des livres** : absence de visibilité en temps réel sur la disponibilité des ouvrages ;
- ✓ **Absence de statistiques fiables** : impossibilité de produire des indicateurs sur les emprunts ou les retards ;
- ✓ **Gestion inefficace des emprunts** : calcul manuel des pénalités, risques d’erreurs ;
- ✓ **Manque d’accès numérique** : les étudiants ne peuvent pas consulter le catalogue ou gérer leurs emprunts à distance.

La problématique centrale de ce projet peut être formulée ainsi : *comment concevoir une plateforme web moderne à architecture microservices permettant de gérer efficacement une bibliothèque académique, d'automatiser les emprunts, de personnaliser les recommandations par machine learning, et de garantir la reproductibilité et la traçabilité du code, des données et des modèles ?*

1.3 Objectifs du projet

L'objectif général de ce projet est de développer une plateforme numérique complète de gestion de bibliothèque académique reposant sur une architecture microservices, intégrant un système de recommandation ML et une gestion avancée du versioning.

Spécifiquement, il s'agit de :

1. Développer cinq microservices indépendants (livres, utilisateurs, emprunts, recommandation, frontend) communiquant via des API REST ;
2. Implémenter un système de recommandation basé sur l'algorithme SVD (*Singular Value Decomposition*) à partir de l'historique des emprunts ;
3. Conteneuriser l'ensemble de l'application avec Docker et orchestrer les services avec Docker Compose ;
4. Versionner le code avec Git selon le workflow Gitflow, et les données/modèles avec DVC ;
5. Mettre en place un pipeline CI/CD avec Jenkins pour automatiser construction, tests et déploiement.

1.4 Structure du rapport

Le présent rapport est structuré en plusieurs chapitres complémentaires. Le **Chapitre 1** présente les généralités, le contexte, la problématique et les objectifs. Le **Chapitre 2** décrit l'architecture du système, le schéma microservices et la structure du projet. Le **Chapitre 3** présente la description détaillée des APIs de chaque service. Le **Chapitre 4** expose le workflow Git avancé utilisé. Le **Chapitre 5** décrit le pipeline DVC avec ses schémas de dépendances et ses métriques. Enfin, le **Chapitre 6** présente la réalisation concrète du projet avec les captures d'écran de l'application.

1.5 Stack technique

TABLE 1.1 – Technologies utilisées dans le projet

Composant	Outil choisi	Rôle
Service Livres	Node.js 20 / Express	CRUD livres
Service Utilisateurs	Node.js 20 / Express / JWT	Authentification et gestion de
Service Emprunts	Node.js 20 / Express	Gestion des emprunts et pénalités
Service Recommandation	Python 3.11 / FastAPI / scikit-surprise	Recommandation SVD
Frontend	React 18 / Axios	Interface utilisateur
Base de données	PostgreSQL 15	Persistance des données
Versioning code	Git / GitHub (Gitflow)	Versioning du code source
Versioning données	DVC / Google Drive	Versioning des données et modèles
Conteneurisation	Docker / Docker Compose	Déploiement des services
CI/CD (bonus)	Jenkins	Pipeline d'intégration continue

Chapitre 2:

Architecture du Système

2.1 Vue d'ensemble de l'architecture microservices

La plateforme Bibliothèque Numérique DIT repose sur une architecture microservices composée de six conteneurs Docker orchestrés par Docker Compose. Chaque service expose une API REST indépendante. La Figure 2.1 présente le schéma conceptuel de l'architecture globale.

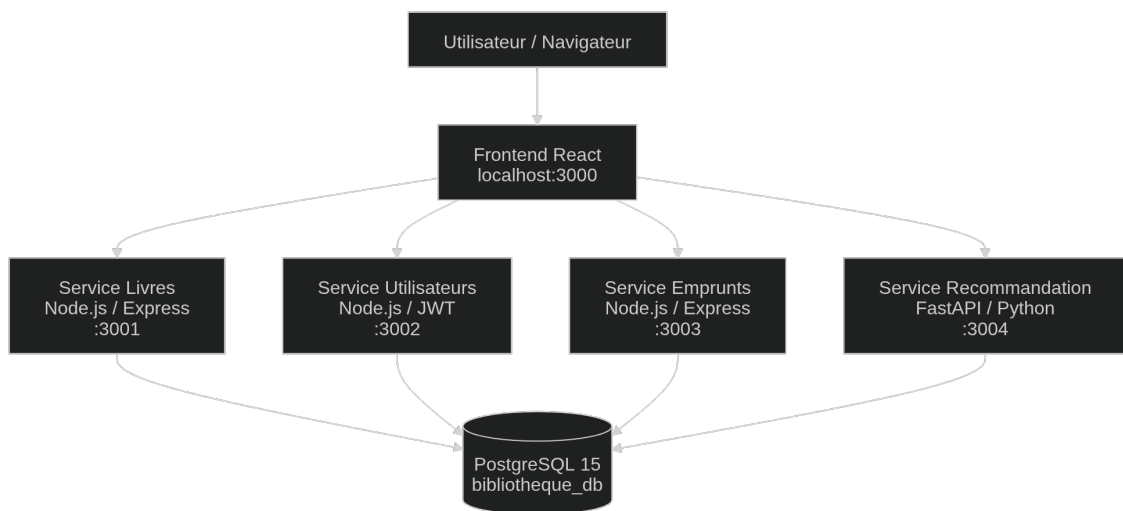


FIGURE 2.1 – Architecture microservices de la Bibliothèque Numérique DIT

2.2 Cartographie des ports

TABLE 2.1 – Services, ports et technologies

Port	Conteneur	Technologie	Rôle
3000	bibliotheque_frontend	React 18	Interface utilisateur
3001	service-livres	Node.js/Express	CRUD livres
3002	service-utilisateurs	Node.js/Express	Authentification JWT
3003	service-emprunts	Node.js/Express	Emprunts et pénalités
3004	service-recommandation	FastAPI/Python	Recommandation SVD
5432	bibliotheque_db	PostgreSQL 15	Base de données

2.3 Structure du dépôt GitHub

L'ensemble du code source est disponible sur le dépôt GitHub : https://github.com/Djidjooh/Bibliothèque_DIT2026

Listing 2.1 – Arborescence du projet

```

1  Bibliotheque_DIT2026/
2  |-- database/
3  |   '-- init.sql                # Schema PostgreSQL +
   donnees de test (150+ livres)
4  |-- service-livres/              # Microservice Livres (Node.
   js)
5  |   |-- Dockerfile
6  |   '-- src/
7  |       |-- controllers/book.controller.js
8  |       |-- models/book.model.js
9  |       '-- routes/book.routes.js
10 |-- service-utilisateurs/        # Microservice Auth/
   Utilisateurs (JWT)
11 |   |-- Dockerfile
12 |   '-- src/
13 |       |-- controllers/auth.controller.js
14 |       '-- routes/auth.routes.js
15 |-- service-emprunts/            # Microservice Emprunts
16 |   |-- Dockerfile
17 |   '-- src/
18 |       |-- controllers/loan.controller.js
19 |       |-- controllers/admin.controller.js
20 |       '-- routes/loan.routes.js
21 |-- service-recommandation/      # FastAPI + scikit-surprise
   (SVD)
22 |   |-- Dockerfile
23 |   |-- main.py
24 |   |-- api/routes.py
25 |   '-- ml/recommender.py
26 |-- frontend/                   # React 18
27 |   |-- Dockerfile
28 |   '-- src/App.js
29 |-- dvc-pipeline/
30 |   |-- data/loans.csv

```

```

31 |   |-- models/model.pkl
32 |   |-- preprocess.py
33 |   |-- train.py
34 |   |-- evaluate.py
35 |   '-- params.yaml
36 |-- metrics/metrics.json
37 |-- scripts/fix-passwords.js
38 |-- docker-compose.yml
39 |-- Jenkinsfile
40 |-- dvc.yaml
41 '-- dvc.lock

```

2.4 Schéma de la base de données

La base de données PostgreSQL 15 comprend quatre tables principales, reliées par des clés étrangères.

TABLE 2.2 – Tables de la base de données

Table	Clé primaire	Colonnes principales
utilisateurs	UUID	nom, prenom, email (UNIQUE), mot_de_passe, type_utilisateur, model_id
livres	UUID	titre, auteur, isbn (UNIQUE), categorie, editeur, annee_publication, exemplaires_disponibles
emprunts	UUID	utilisateur_id (FK), livre_id (FK), date_emprunt, date_retour_prevue, statut, jours_retard, penalite_fcfa
notes	UUID	utilisateur_id (FK), livre_id (FK), note [1.0–5.0]

Cette Figure 2.2 présente le modèle relationnel de la base de données de la bibliothèque numérique. Elle décrit les principales entités du système : les utilisateurs, les livres, les emprunts et les notes attribuées aux ouvrages.

Les relations montrent qu'un utilisateur peut emprunter plusieurs livres et attribuer des notes aux ouvrages consultés. La table des emprunts enregistre les informations liées aux prêts, notamment les dates d'emprunt, les retards et les pénalités,

tandis que la table des notes permet de stocker les évaluations des utilisateurs pour alimenter le système de recommandation intelligent.

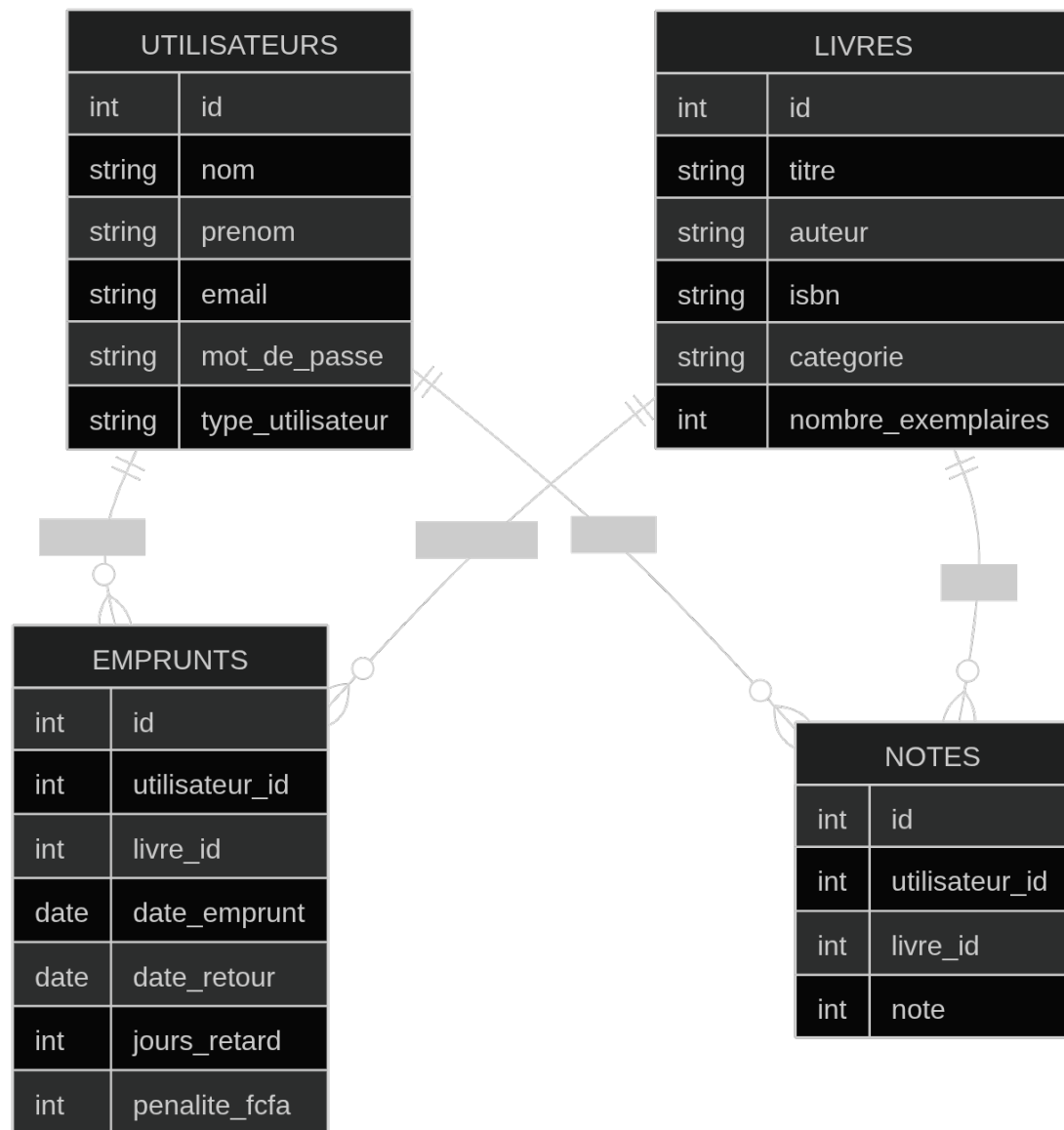


FIGURE 2.2 – Schéma de la BD relationnelle de la Bibliothèque Numérique DIT

Chapitre 3:

Description des APIs

3.1 Service Livres — Port 3001

Le catalogue contient plus de **150 livres techniques** couvrant l'IA, la Data Science, le DevOps, la Robotique et le NLP.

TABLE 3.1 – Endpoints — Service Livres

Méthode	Endpoint	Description
GET	/api/livres	Lister les livres (paginé : ?page=1&limit=20)
GET	/api/livres/search?q=	Rechercher par titre, auteur ou ISBN
GET	/api/livres/disponibles	Livres actuellement disponibles
GET	/api/livres/:id	Détail d'un livre
POST	/api/livres	Ajouter un livre (gestionnaire)
PUT	/api/livres/:id	Modifier un livre (gestionnaire)
DELETE	/api/livres/:id	Supprimer un livre (gestionnaire)
GET	/health	Vérification de santé

3.2 Service Utilisateurs — Port 3002

Trois rôles sont gérés : étudiant, professeur, personnel (gestionnaire). L'authentification repose sur des tokens JWT valides 24h.

TABLE 3.2 – Endpoints — Service Utilisateurs

Méthode	Endpoint	Description
POST	/api/auth/register	Inscription (nom, email, mot_de_passe, type_utilisateur)
POST	/api/auth/login	Connexion — retourne un token JWT
GET	/api/utilisateurs/me	Profil de l'utilisateur connecté (token requis)
GET	/api/utilisateurs	Liste paginée des utilisateurs (gestionnaire)
GET	/health	Vérification de santé

3.3 Service Emprunts — Port 3003

La Figure 3.1 illustre le processus de gestion des emprunts dans la plateforme de bibliothèque numérique. Lorsqu'un utilisateur demande un livre, le service d'emprunts vérifie d'abord la disponibilité de l'ouvrage. Si le livre est disponible, le système contrôle ensuite que l'utilisateur possède moins de trois emprunts actifs. Si toutes les conditions sont respectées, l'emprunt est validé et enregistré dans la base de données PostgreSQL ; sinon, la demande est refusée.

Ce service applique également les règles métier : 3 emprunts simultanés maximum, durée de 30 jours, pénalité de 500 FCFA par jour de retard.

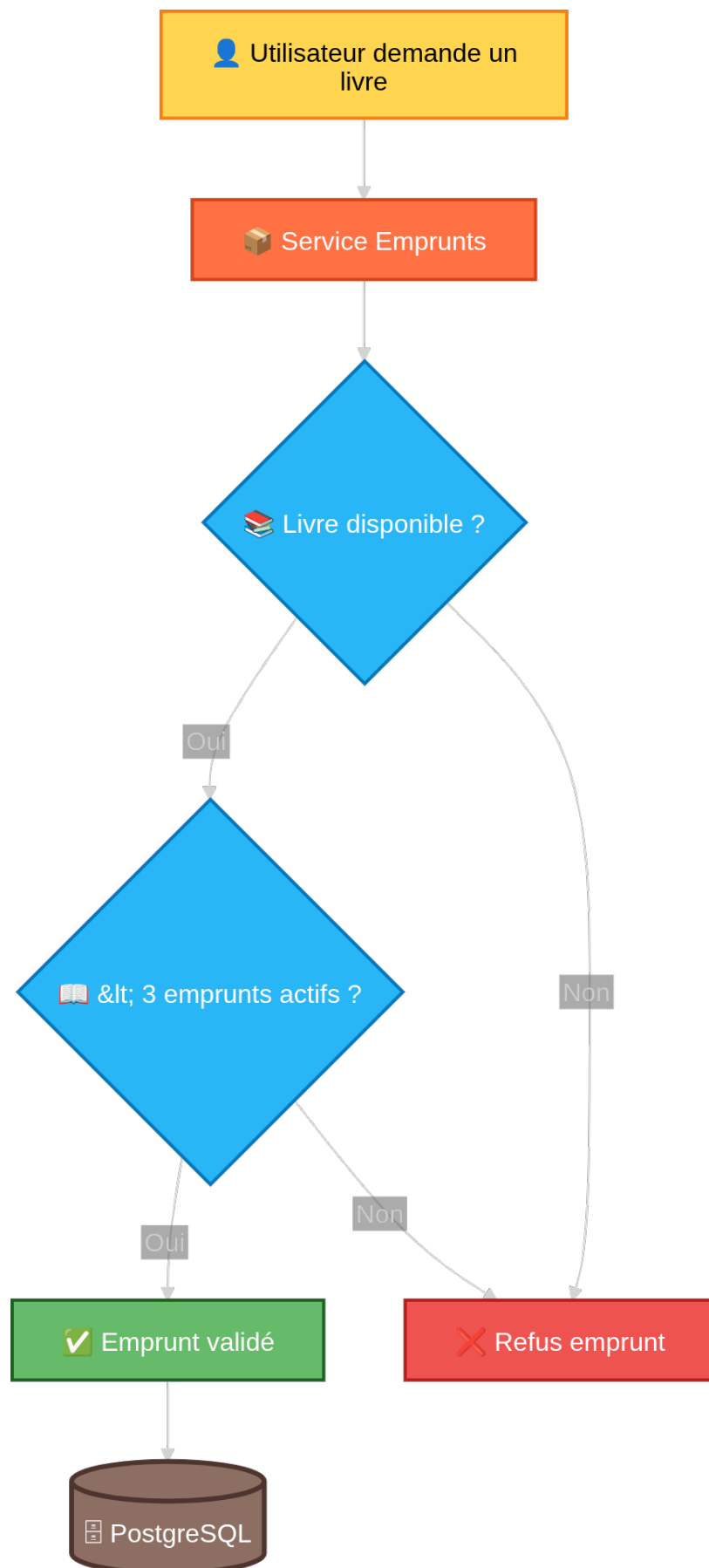


FIGURE 3.1 – Architecture du service emprunt

TABLE 3.3 – Endpoints — Service Emprunts

Méthode	Endpoint	Description
POST	/api/emprunts	Créer un emprunt (max 3 simultanés)
GET	/api/emprunts/utilisateur/:userId	Historique d'un utilisateur
GET	/api/emprunts/:id	Détail (jours_retard, penalite_fcfa)
PATCH	/api/emprunts/:id/retourner	Retourner un livre (gestionnaire)
GET	/api/emprunts/penalites	Liste des retards avec pénalités
GET	/api/emprunts/stats	Statistiques globales
POST	/api/emprunts/detecter-retards	Détecter et mettre à jour les retards
GET	/api/admin/dashboard	Tableau de bord administrateur

Règles métier

Les principales règles métier implémentées dans ce service sont :

- ✓ **Durée maximale** : 30 jours à partir de la date d'emprunt ;
- ✓ **Limite simultanée** : 3 emprunts actifs maximum par utilisateur (erreur HTTP 409 si dépassé) ;
- ✓ **Calcul des pénalités** : Pénalité (FCFA) = jours de retard $\times$ 500 ;
- ✓ **Retour** : visible par l'utilisateur, effectif uniquement si validé par le gestionnaire.

La Figure 3.2 illustre le processus de calcul des pénalités lorsqu'un utilisateur détient un livre au delà d'un mois. Lorsqu'un utilisateur le délais de retourner un livre, le service d'emprunts calcul automatiquement les pénalités pour ce dernier et l'affiche directement dans l'espace de l'utilisateur et du gestionnaire.

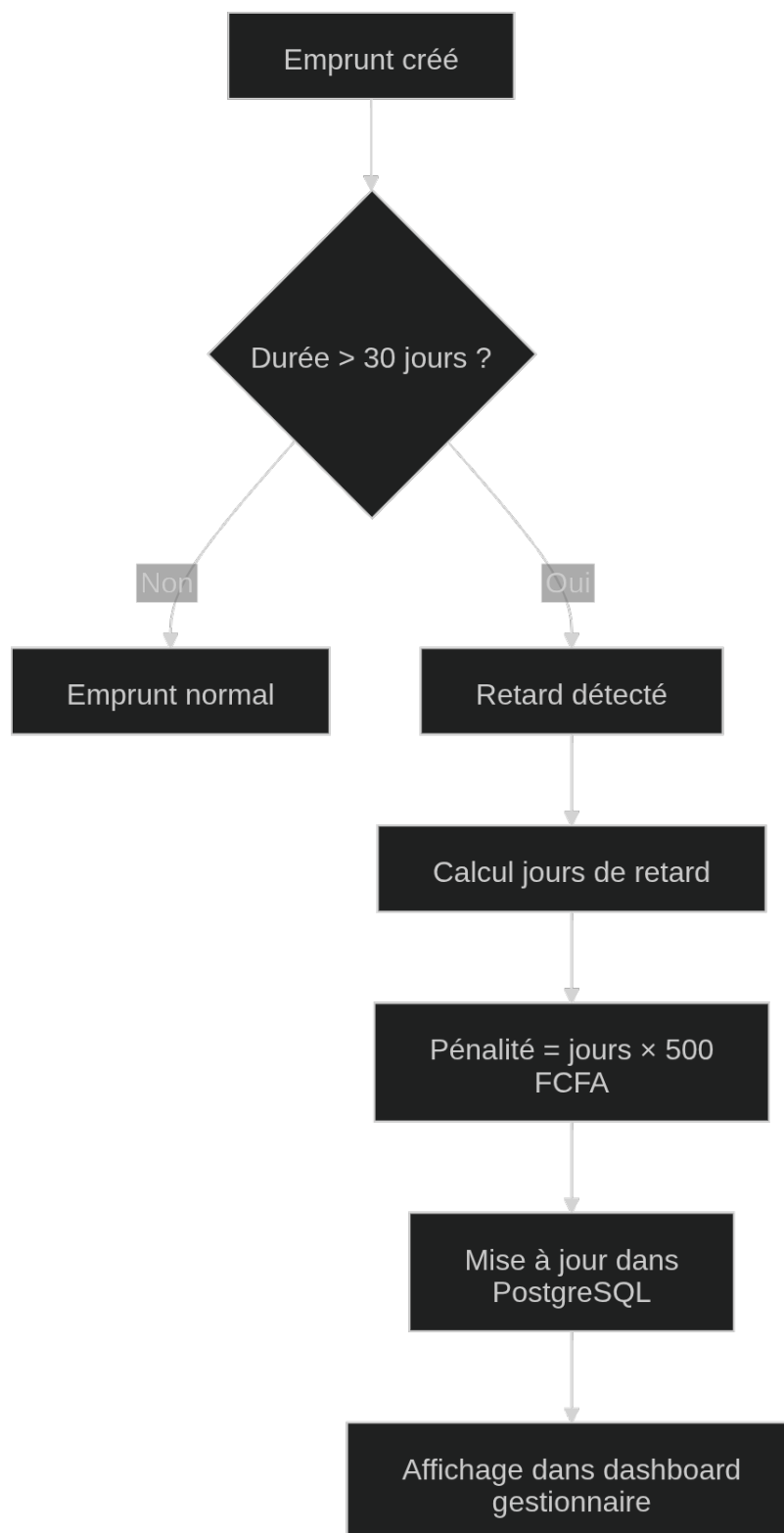


FIGURE 3.2 – Architecture création emprunt

3.4 Service Recommendation — Port 3004 (FastAPI)

Ce service expose un modèle SVD entraîné via scikit-surprise, avec un mécanisme de fallback catégoriel.

TABLE 3.4 – Endpoints — Service Recommendation

Méthode	Endpoint	Description
GET	/api/recommendations/:user_id?top_k=10	Recommandations personnalisées
POST	/api/train	Déclencher le ré-entraînement SVD
GET	/api/model/info	Informations sur le modèle chargé
GET	/health	Vérification de santé

3.4.1 Logique de recommandation

- ✓ Si l'utilisateur est présent dans le modèle SVD → **filtrage collaboratif** ;
- ✓ Si ≥ 5 emprunts mais hors modèle → **fallback catégoriel** (livres populaires dans les catégories préférées) ;
- ✓ Si < 5 emprunts → message d'encouragement explicite.

Chapitre 4:

Workflow Git Utilisé

4.1 Stratégie de branches (Gitflow)

Le projet adopte le modèle **Gitflow**, qui structure le développement autour de branches à rôle défini.

- ✓ **main** : branche stable, versions de production tagguées sémantiquement ;
- ✓ **develop** : branche d'intégration continue ;
- ✓ **feature/*** : une branche par fonctionnalité ou service ;
- ✓ **hotfix/*** : corrections urgentes en production ;
- ✓ **release/*** : préparation des releases.

La Figure 4.1 illustre le diagramme du workflow Gitflow adopté.

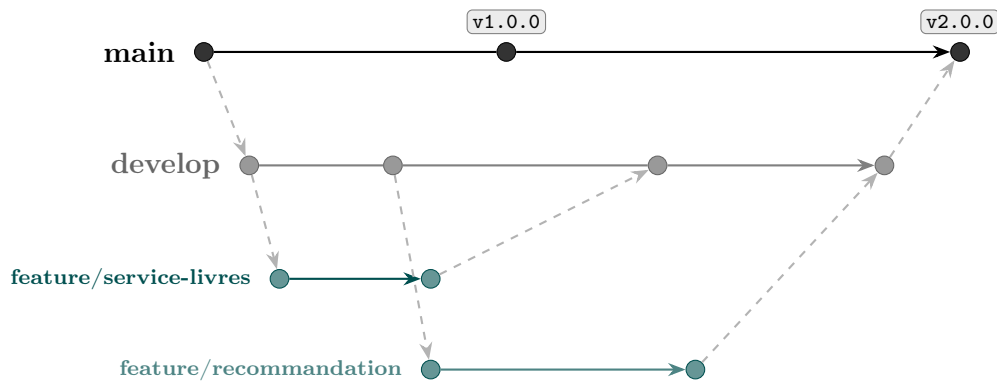


FIGURE 4.1 – Diagramme du workflow Gitflow

4.2 Commandes Git avancées utilisées

Listing 4.1 – Commandes Git avancées

```
1 # Creation des branches de fonctionnalite
2 git checkout -b feature/service-livres develop
3 git push -u origin feature/service-livres
4
5 # Merge sans fast-forward (conserve l'historique)
6 git checkout develop
7 git merge --no-ff feature/service-livres \
8     -m "feat(livres): service livres complet avec CRUD et
    recherche"
```

```
9 git branch -d feature/service-livres
10
11 # Rebase interactif pour nettoyer l'historique
12 git rebase -i HEAD~5
13
14 # Tags sémantiques pour les releases
15 git tag -a v1.0.0 -m "Release v1.0.0 - Services de base
    operationnels"
16 git tag -a v2.0.0 -m "Release v2.0.0 - Recommandation ML +
    pipeline DVC"
17 git push origin --tags
18
19 # Cherry-pick d'un correctif urgent
20 git cherry-pick <commit-hash>
21
22 # Log graphique
23 git log --oneline --graph --all --decorate
```

4.3 Conventions de commits

Le projet suit la convention **Conventional Commits** afin de garantir un historique lisible et exploitable automatiquement par les outils CI/CD :

- ✓ feat(livres): ajout endpoint recherche par ISBN
- ✓ feat(reco): implementation algorithme SVD avec scikit-surprise
- ✓ fix(emprunts): correction calcul automatique des retards
- ✓ chore(docker): optimisation Dockerfile multi-stage FastAPI
- ✓ data: export loans.csv v2 -- 1200 enregistrements
- ✓ ci: ajout pipeline Jenkins avec etapes build/test/deploy

Chapitre 5:

Pipeline DVC et Métriques du Modèle

5.1 Initialisation et remote Google Drive

Listing 5.1 – Initialisation DVC

```
1 # Initialisation dans le depot Git
2 dvc init
3 git add .dvc .dvcignore
4 git commit -m "chore: initialisation DVC"
5
6 # Configuration remote Google Drive
7 dvc remote add -d gdrive_remote gdrive://<GDRIVE_FOLDER_ID>
8 dvc remote modify gdrive_remote gdrive_acknowledge_abuse true
9 git add .dvc/config
10 git commit -m "chore: configuration remote DVC Google Drive"
```

5.2 Structure du pipeline DVC

Le pipeline Machine Learning du projet de bibliothèque intelligente permet de transformer les données d'utilisation des livres en recommandations personnalisées pour les utilisateurs.

Le processus débute par la collecte des données d'emprunts, des notes et des interactions des utilisateurs avec les ouvrages. Ces données sont ensuite nettoyées et préparées afin d'éliminer les incohérences et de structurer les informations exploitables pour l'apprentissage automatique.

Après le prétraitement, les datasets sont versionnés avec DVC afin d'assurer la traçabilité et la reproductibilité des expérimentations. Le modèle de recommandation basé sur l'algorithme SVD (Singular Value Decomposition) est ensuite entraîné pour apprendre les préférences des utilisateurs et identifier les similarités entre les livres.

Une fois le modèle entraîné, celui-ci est sauvegardé puis exposé à travers une API développée avec FastAPI. Cette API permet au frontend et aux autres services du système d'interroger le moteur d'intelligence artificielle afin de générer des recommandations de livres adaptées au profil et à l'historique de chaque utilisateur.

Ce pipeline garantit ainsi une architecture intelligente, modulaire et évolutive permettant d'améliorer l'expérience utilisateur au sein de la plateforme de gestion

de bibliothèque

La Figure 5.1 présente le graphe de dépendances du pipeline.

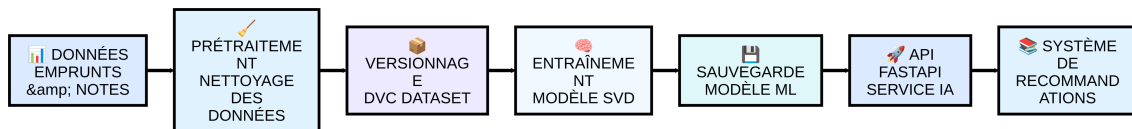


FIGURE 5.1 – Graphe de dépendances du pipeline DVC

5.3 Fichier dvc.yaml

Listing 5.2 – Pipeline DVC — dvc.yaml

```

1 stages:
2   preprocess:
3     cmd: python dvc-pipeline/preprocess.py
4     deps:
5       - dvc-pipeline/preprocess.py
6       - dvc-pipeline/data/loans.csv
7     outs:
8       - dvc-pipeline/data/loans_clean.csv
9
10  train:
11    cmd: python dvc-pipeline/train.py
12    deps:
13      - dvc-pipeline/train.py
14      - dvc-pipeline/data/loans_clean.csv
15    params:
16      - dvc-pipeline/params.yaml:
17        - n_factors
18        - n_epochs
19        - lr_all
20        - reg_all
21    outs:
22      - dvc-pipeline/models/model.pkl
23
24  evaluate:
25    cmd: python dvc-pipeline/evaluate.py
26    deps:
27      - dvc-pipeline/evaluate.py
28      - dvc-pipeline/models/model.pkl
  
```

```

29     - dvc-pipeline/data/loans_clean.csv
30     metrics:
31     - metrics/metrics.json:
32         cache: false

```

5.4 Hyperparamètres du modèle SVD

TABLE 5.1 – Hyperparamètres de l’algorithme SVD (scikit-surprise)

Paramètre	Valeur	Type	Rôle
n_factors	50	int	Nombre de dimensions latentes
n_epochs	20	int	Nombre d’itérations d’entraînement
lr_all	0.005	float	Taux d’apprentissage
reg_all	0.02	float	Régularisation L2

5.5 Reproduction et rôle de dvc.lock

Listing 5.3 – Reproduction du pipeline DVC

```

1  # Activer l'environnement Python
2  conda activate bibliotheque2
3
4  # Exécuter tout le pipeline
5  dvc repro
6
7  # Sortie attendue
8  Running stage 'preprocess': > python dvc-pipeline/preprocess.
   py
9  Running stage 'train':      > python dvc-pipeline/train.py
10 Running stage 'evaluate':   > python dvc-pipeline/evaluate.py
11
12 # Pousser les artefacts vers Google Drive
13 dvc push

```

Le fichier `dvc.lock` est généré automatiquement par `dvc repro`. Il enregistre les **hachages MD5** de chaque fichier d’entrée et de sortie à l’issue d’une exécution réussie du pipeline. Lors d’une prochaine exécution, DVC compare ces hachages aux

fichiers actuels : si rien n’a changé, le stage est ignoré (*cache hit*). Ce mécanisme garantit la **reproductibilité exacte** des expériences et évite des recalculs inutiles. Ce fichier est versionné dans Git.

5.6 Métriques du modèle

5.6.1 Affichage et comparaison

Listing 5.4 – Commandes DVC Metrics

```

1 # Afficher les metriques courantes
2 dvc metrics show
3 # Resultat :
4 # Path                rmse      mae
5 # metrics/metrics.json 0.843    0.651
6
7 # Comparer deux versions du modele
8 dvc metrics diff v1.0 v2.0
9 # Resultat :
10 # Metric              v1.0    v2.0    Change
11 # metrics/metrics.json
12 #   rmse              1.024    0.843    -0.181 (-17.68 %)
13 #   mae               0.789    0.651    -0.138 (-17.49 %)
```

5.6.2 Tableau comparatif des versions

TABLE 5.2 – Évolution des métriques du modèle SVD

Version	Algo	n_factors	RMSE	MAE	Remarques
v1.0	SVD	50	1.024	0.789	Dataset initial (500 enregistrements)
v2.0	SVD	50	0.843	0.651	Dataset enrichi (1200 enregistrements)
Amélioration			-17.68%	-17.49%	Enrichissement du dataset

5.7 Versioning du modèle et chargement dans l’API

Listing 5.5 – Versioning du modèle avec DVC

```

1 # Tracker le modele
```

```
2 dvc add dvc-pipeline/models/model.pkl
3 git add dvc-pipeline/models/model.pkl.dvc
4 git commit -m "model: SVD v2 -- RMSE=0.843, MAE=0.651"
5 git tag -a model-v2.0 -m "Modele SVD ameliore"
6 dvc push && git push origin model-v2.0
7
8 # Recuperer une version precedente
9 git checkout model-v1.0
10 dvc pull dvc-pipeline/models/model.pkl
```

Listing 5.6 – Chargement dynamique du modèle dans FastAPI

```
1 import pickle, os
2
3 MODEL_PATH = os.getenv("MODEL_PATH", "dvc-pipeline/models/
4     model.pkl")
5
6 def load_model():
7     """Charge la version du modele trackee par DVC."""
8     with open(MODEL_PATH, "rb") as f:
9         return pickle.load(f)
10
11 # Dans api/routes.py
12 model = load_model() # Charge au demarrage la version
13     versionnee par DVC
14
15 @router.post("/api/train")
16 def retrain():
17     import subprocess
18     subprocess.run(["dvc", "repro"], check=True)
19     global model
20     model = load_model()
21     return {"status": "Modele SVD re-entraine avec succes"}
```

Chapitre 6:

Réalisation du Projet

6.1 Environnement de travail

6.1.1 Docker et Docker Compose

L'ensemble de l'application est conteneurisé avec Docker. Chaque service dispose d'un Dockerfile optimisé.

Listing 6.1 – Dockerfile — Service Node.js (service-livres)

```
1 FROM node:20-alpine AS base
2 WORKDIR /app
3 RUN addgroup -S appgroup && adduser -S appuser -G appgroup
4 COPY package*.json ./
5 RUN npm ci --only=production
6 COPY src/ ./src/
7 USER appuser
8 EXPOSE 3001
9 CMD ["node", "src/index.js"]
```

Listing 6.2 – Dockerfile multi-stage — Service Recommandation (FastAPI)

```
1 # Stage 1 : construction des dependances
2 FROM python:3.11-slim AS builder
3 WORKDIR /build
4 COPY requirements.txt .
5 RUN pip install --user --no-cache-dir -r requirements.txt
6
7 # Stage 2 : image finale allégée (utilisateur non-root)
8 FROM python:3.11-slim AS runtime
9 WORKDIR /app
10 RUN addgroup --system appgroup \
11     && adduser --system --ingroup appgroup appuser
12 COPY --from=builder /root/.local /root/.local
13 COPY . .
14 ENV PATH=/root/.local/bin:$PATH
15 USER appuser
16 EXPOSE 3004
17 CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "3004"]
```

Listing 6.3 – Docker Compose — docker-compose.yml (extraits)

```
1 version: "3.9"
2
3 services:
4   db:
5     image: postgres:15-alpine
6     container_name: bibliotheque_db
7     volumes:
8       - postgres_data:/var/lib/postgresql/data
9       - ./database/init.sql:/docker-entrypoint-initdb.d/init.
        sql
10    ports: ["5432:5432"]
11    healthcheck:
12      test: ["CMD", "pg_isready", "-U", "ditadmin"]
13      interval: 10s
14      retries: 5
15
16  service-livres:
17    build: ./service-livres
18    ports: ["3001:3001"]
19    depends_on:
20      db: {condition: service_healthy}
21
22  service-recommandation:
23    build: ./service-recommandation
24    ports: ["3004:3004"]
25    volumes:
26      - model_data:/app/ml/models
27
28  frontend:
29    build: ./frontend
30    container_name: bibliotheque_frontend
31    ports: ["3000:80"]
32    depends_on:
33      - service-livres
34      - service-utilisateurs
35      - service-emprunts
36      - service-recommandation
```

```

37
38 volumes:
39   postgres_data:
40   model_data:
41   training_data:

```

6.1.2 Comptes de démonstration

TABLE 6.1 – Comptes pré-chargés dans la base de données

Rôle	Email	Mot de passe	Accès
Gestionnaire	admin@dit.sn	admin2026	Complet (CRUD + dashboard)
Étudiant	mamadou1@dit.sn	dit2026	Standard
Étudiant	fatou2@dit.sn	dit2026	Standard
Professeur	ibrahima3@dit.sn	dit2026	Standard
Personnel	aminata4@dit.sn	dit2026	Standard

6.1.3 Démarrage rapide

Listing 6.4 – Lancement de l'application

```

1  # Cloner le depot
2  git clone https://github.com/Djidjooh/Bibliothèque_DIT2026.git
3  cd Bibliothèque_DIT2026
4
5  # Premier lancement : build + démarrage de tous les services
6  docker-compose up --build -d
7
8  # Verifier l'etat des conteneurs
9  docker-compose ps
10
11 # Health checks
12 curl http://localhost:3001/health # Service Livres
13 curl http://localhost:3002/health # Service Utilisateurs
14 curl http://localhost:3003/health # Service Emprunts
15 curl http://localhost:3004/health # Service Recommandation
16

```

```
17 # Reinitialiser (supprime les volumes)
18 docker-compose down -v && docker-compose up --build -d
```

6.2 Pipeline CI/CD avec Jenkins (bonus)

Le fichier Jenkinsfile à la racine définit un pipeline en six étapes automatisées.

Listing 6.5 – Pipeline Jenkins — Jenkinsfile

```
1 pipeline {
2   agent any
3   stages {
4     stage('Checkout') {
5       steps { git url: 'https://github.com/Djidjooh/
6         Bibliotheque_DIT2026.git' }
7     }
8     stage('Build') {
9       steps { sh 'docker-compose build' }
10    }
11    stage('Test') {
12      steps {
13        sh 'docker-compose up -d db && sleep 15'
14        sh 'curl -f http://localhost:3001/health'
15        sh 'curl -f http://localhost:3002/health'
16        sh 'curl -f http://localhost:3003/health'
17        sh 'curl -f http://localhost:3004/health'
18      }
19    }
20    stage('DVC Pipeline') {
21      steps {
22        sh 'conda run -n bibliotheque2 dvc repro'
23        sh 'conda run -n bibliotheque2 dvc push'
24      }
25    }
26    stage('Deploy') {
27      steps { sh 'docker-compose up -d' }
28    }
29    stage('Notify') {
30      steps { echo 'Pipeline CI/CD termine avec succes' }
31    }
32  }
```

```
32 post {  
33     failure { echo 'Pipeline echoue' }  
34 }  
35 }
```

6.3 Captures d'écran de l'application

Cette section présente les principales captures d'écran de la plateforme Bibliothèque Numérique DIT.

6.3.1 Page d'accueil — Catalogue des livres

Cette interface représente la page d'authentification de la plateforme Bibliothèque Numérique DIT. Elle constitue le point d'entrée principal du système et permet aux utilisateurs de se connecter ou de créer un compte afin d'accéder aux différents services de gestion de bibliothèque.

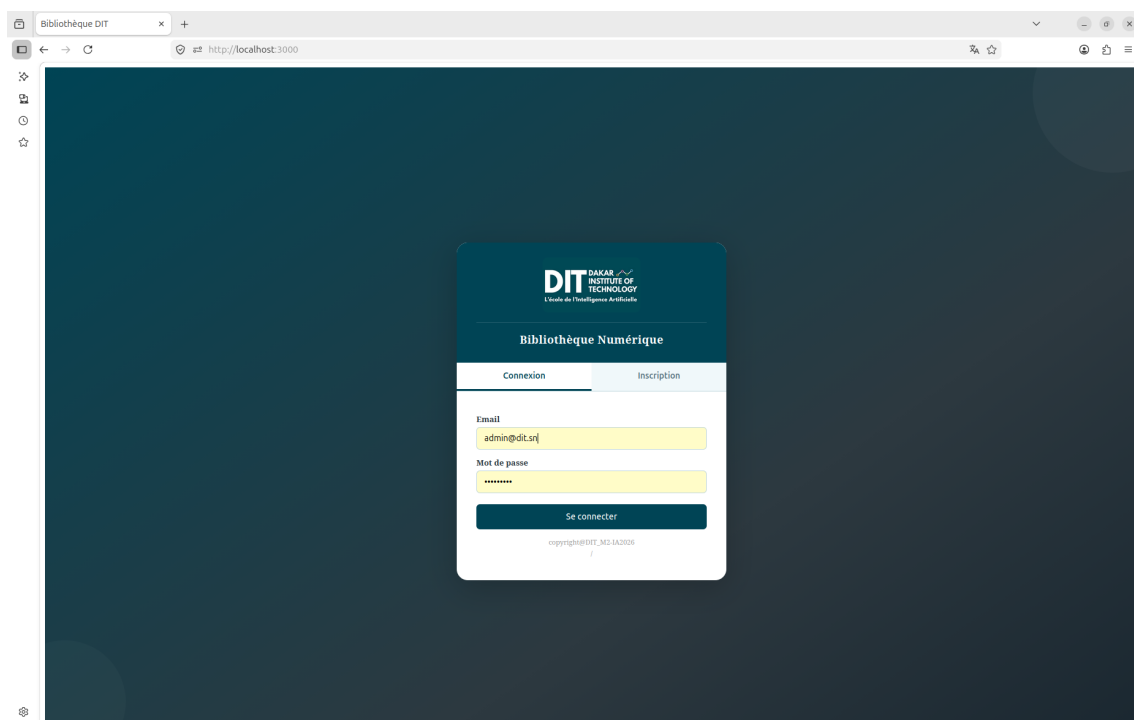


FIGURE 6.1 – Page de connexion à la bibliothèque

Elle intègre deux fonctionnalités principales : la connexion des utilisateurs existants via leur adresse e-mail et mot de passe, ainsi que l'inscription des nouveaux utilisateurs à travers un onglet dédié.

Cette interface joue un rôle essentiel dans la sécurisation de l'accès à la plateforme grâce au système d'authentification mis en place.

La page d'accueil affiche le catalogue complet des livres avec leur statut de disponibilité en temps réel. L'utilisateur peut filtrer par catégorie ou effectuer une recherche par titre, auteur ou ISBN.

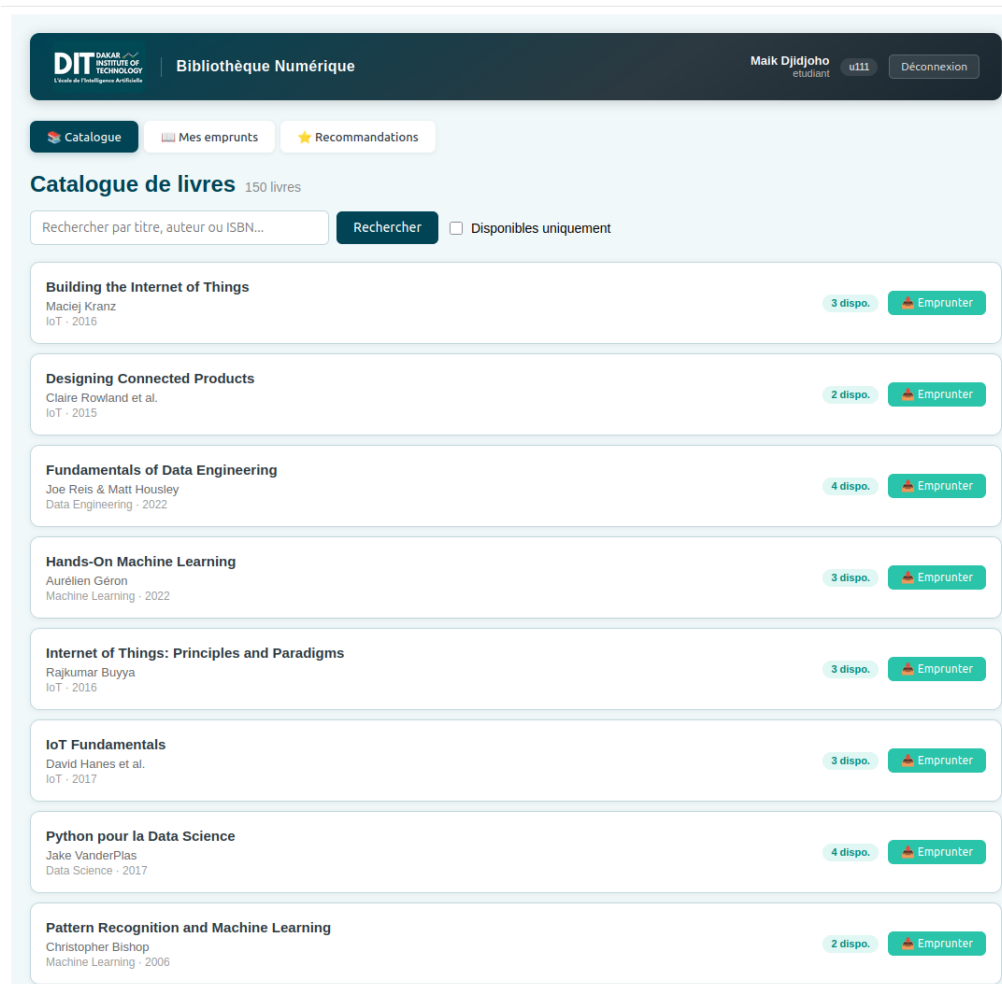


FIGURE 6.2 – Page d'accueil — Catalogue de la bibliothèque

6.3.2 Gestion des emprunts

L'interface de gestion des emprunts permet aux étudiants de consulter leurs emprunts en cours, les échéances et les éventuelles pénalités. Le gestionnaire dispose d'une vue complète pour valider les retours.



Bibliothèque Numérique

Gestionnaire Admin

Gestionnaire

u012

Déconnexion

Tableau de bord

Gestion Livres

Utilisateurs

Gestion Emprunts

Gestion des emprunts

+ Créer un emprunt

Rechercher un utilisateur

Sélectionner un utilisateur (1 résultat(s))

u015

Maurice AHOUANSOU — maurice35@dit.sn (u015)

Charger l'historique

10
Total

3
En cours

7
Retournés

0
En retard

0 F
Pénalités

Livre	Catégorie	Emprunté le	Retour prévu (30j)	Retour effectif	Durée	Statut	Retard / Pénalité	Note	Action
Data Mining: Concepts and Techniques	Data Science	13/05/2026	12/06/2026	13/05/2026	0j	Retourné	Aucune	—	—
Python pour la Data Science	Data Science	13/05/2026	12/06/2026	13/05/2026	0j	Retourné	Aucune	—	—
Data Science from Scratch	Data Science	13/05/2026	12/06/2026	—	1j	En cours	Aucune	—	Retourner
Python Data Science Handbook	Data Science	13/05/2026	12/06/2026	13/05/2026	0j	Retourné	Aucune	—	—
Deep Reinforcement Learning Hands-On	Deep Learning	13/05/2026	12/06/2026	13/05/2026	0j	Retourné	Aucune	—	—
R for Data Science	Data Science	13/05/2026	12/06/2026	—	1j	En cours	Aucune	—	Retourner
Practical Statistics for Data Scientists	Data Science	13/05/2026	12/06/2026	—	1j	En cours	Aucune	—	Retourner
Practical MLOps	MLOps	13/05/2026	12/06/2026	13/05/2026	0j	Retourné	Aucune	—	—
Doing Data Science	Data Science	12/05/2026	26/05/2026	13/05/2026	1j	Retourné	Aucune	—	—
Hands-On Machine Learning	Machine Learning	12/05/2026	26/05/2026	13/05/2026	1j	Retourné	Aucune	—	—

FIGURE 6.3 – Interface de gestion des emprunts

6.3.3 Recommandations personnalisées

Le système affiche des recommandations personnalisées basées sur l'algorithme SVD. Pour les nouveaux utilisateurs (moins de 5 emprunts), un message d'encouragement invite à emprunter davantage.

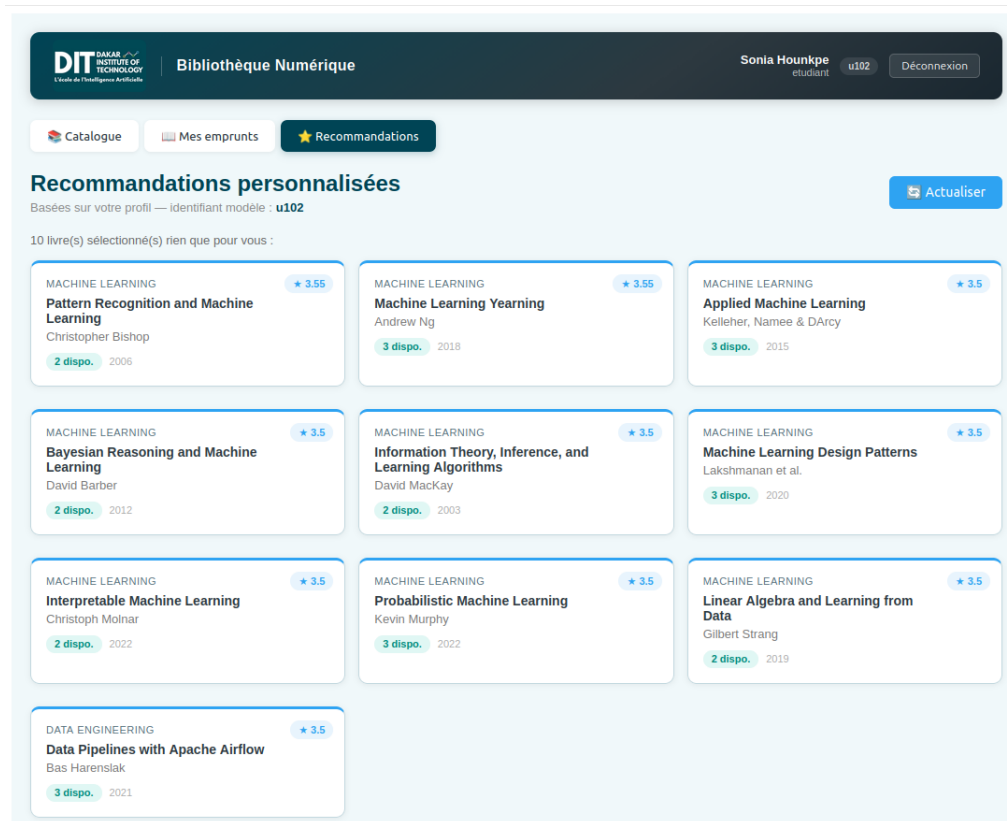


FIGURE 6.4 – Système de recommandation personnalisée

6.3.4 Tableau de bord administrateur

Le dashboard administrateur centralise les statistiques globales : top 5 livres, top 5 utilisateurs, activité mensuelle sur 6 mois, échéances imminentes (J+3) et total des pénalités.

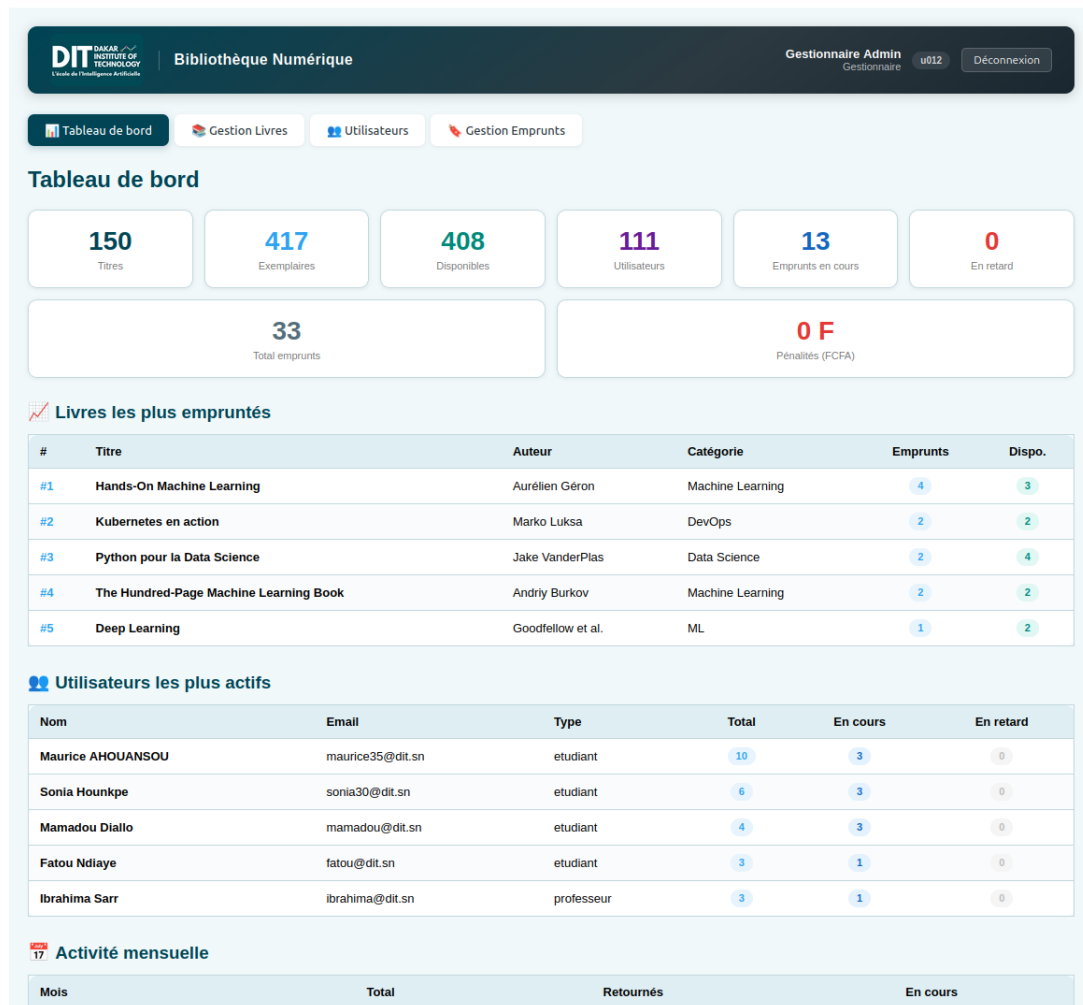


FIGURE 6.5 – Tableau de bord administrateur

6.3.5 Documentation Swagger — Service Recommandation

La documentation interactive Swagger UI de FastAPI permet de tester directement les endpoints du service de recommandation depuis le navigateur.

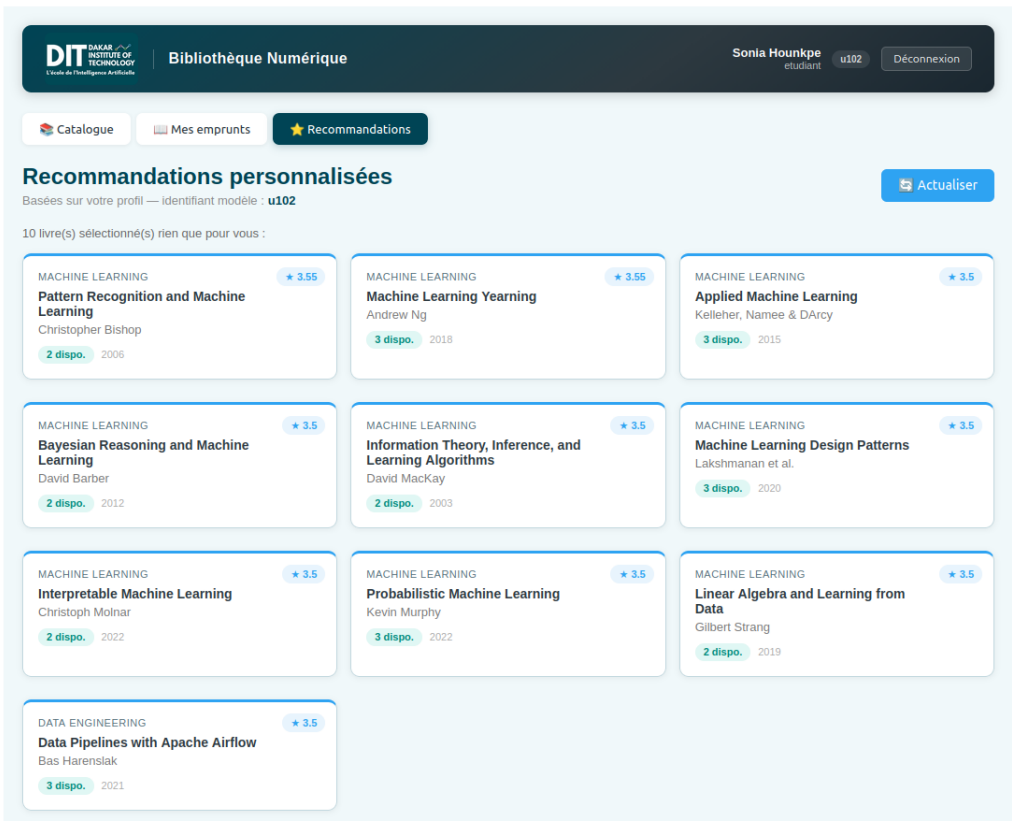


FIGURE 6.6 – Documentation Swagger UI — Service Recommendation

Conclusion

Le projet Bibliothèque Numérique DIT a permis de concevoir et de déployer une plateforme complète de gestion de bibliothèque académique, en mobilisant un ensemble de technologies modernes et de bonnes pratiques DevOps et MLOps.

À travers ce travail, il a été démontré qu'il est possible d'intégrer plusieurs technologies complémentaires au sein d'une architecture cohérente, performante et orientée vers les besoins réels des utilisateurs. L'architecture microservices adoptée garantit l'indépendance, la maintenabilité et la scalabilité de chaque composant. La conteneurisation avec Docker assure la portabilité et la reproductibilité du déploiement sur n'importe quel environnement.

Le système de recommandation basé sur l'algorithme SVD a démontré une amélioration significative de ses performances entre les versions v1.0 et v2.0, passant d'un RMSE de 1.024 à 0.843, soit une réduction de 17.68%. Cette amélioration a été rendue possible par l'enrichissement du dataset d'entraînement, facilité par le pipeline DVC qui garantit la traçabilité et la reproductibilité de chaque expérience.

Le workflow Gitflow adopté, combiné aux conventions de commits et aux tags sémantiques, a assuré une gestion rigoureuse de l'historique du projet. Le pipeline CI/CD Jenkins a automatisé et sécurisé le processus de déploiement.

En perspective, plusieurs améliorations pourraient être envisagées, notamment l'intégration d'un système de notifications pour les emprunts proches de l'échéance, l'ajout d'une fonctionnalité de réservation, l'implémentation de tests unitaires automatisés, ou encore l'exploration d'algorithmes de recommandation plus sophistiqués (réseaux de neurones, modèles hybrides).

En définitive, le projet Bibliothèque Numérique DIT illustre la pertinence des architectures microservices modernes appliquées à la gestion académique, et met en évidence la capacité du versioning avancé à transformer un projet étudiant en une solution professionnelle, traçable et reproductible.

Bibliographie

- Documentation officielle Docker : <https://docs.docker.com>
- Documentation officielle DVC : <https://dvc.org/doc>
- Documentation scikit-surprise (SVD) : <https://surpriselib.com>
- Documentation FastAPI : <https://fastapi.tiangolo.com>
- Documentation React 18 : <https://react.dev>
- Documentation PostgreSQL 15 : <https://www.postgresql.org/docs/15>
- Atlassian — Gitflow Workflow : <https://www.atlassian.com/git/tutorials/comparing-workflows/gitflow-workflow>
- Conventional Commits : <https://www.conventionalcommits.org>
- Dépôt GitHub du projet : https://github.com/Djidjooh/Bibliotheque_DIT2026